05 September 2026

Organised By : Data Science Club

ORIGIN'26 HACKATHON

Presented By :

Team Zero/Zero

SETTLEMENT Q&A AGENT

Team Members :

Utkarsh	25BAI11231
Kartikey Srivastava	25BAI11235
Atharva Kulkarni	25BAI10584

The Problem

What happens when a merchant asks:

"Why wasn't my settlement processed?"

Today, answering this question requires checking multiple systems such as :

- Payment Gateway Records
- Bank Settlement Records
- Internal Ledger Records
- Transaction Timestamps
- **Failure/ Exception**

This results in :

Manual investigation → Time-consuming → Error-prone → Delayed response

Key Challenge to this Settlement is:

The merchant should not have to understand or manually inspect these technical systems.

The Solution Our Team has Implemented

- **Settlement Q&A Agent** is an AI-powered Agent which works as an automated settlement operations employee for the merchant.
- The merchant simply provides: Transaction ID
- The system automatically :

    Finds the transaction
    Traces it across Gateway, Bank & Ledger
    Reconciles the records
    Detects anomalies and missing evidence
    Determines the most defensible settlement state
    Explains the result in plain English

What is the Core Principle of our Solution?

- The Core Principle is:

  ‘The backend determines the settlement status from verified transaction data first.’
- The system then uses AI to explain the findings in simple and merchant-friendly language.
- This ensures the AI does not invent transaction facts and clearly reports uncertainty or exceptions.
- Here, the AI does not invent the “Transaction Status”.

DATA FIRST → DECISION SECOND → AI EXPLANATION LAST

System Architecture and Workflow

Merchant/ Business Owner

↓

Transaction API + Transaction Service

↓

Gateway + Bank + Ledger Data

↓

Reconciliation and Anomaly Service

↓

Investigation and LLM Service

↓

Merchant-Friendly Reply

What is the Key Principle of the Workflow?

The backend decides from verified data.
AI explains the result.

The system traces the transaction across the three sources, compares the evidence, detects anomalies, forms a structured investigation result, and finally generates a plain-English explanation.

Data & Decision Engine

We have used three Data Sources:

Source	Purpose
Gateway	Payment Transaction and Gateway Status
Bank	Settlement Status and Reason for Exceptions
Ledger	Provides Internal Accounting Status

Common Key **transaction_id** is maintained across all datasets for tracing.

Possible Outcomes include:

- Success/ Settled
- Pending
- Delayed
- Failed
- Mismatch (Exception)
- Unknown (Exception)

Investigation Example

Our project uses mock CSV data with realistic fields and also includes both normal and problematic scenarios for testing.

Transaction ID: TXN-000005

System	Result
Payment Gateway	Success
Bank	Settlement_Failed
Ledger	Pending

Final Status : Delayed

The payment was successfully processed by the gateway, but the bank settlement failed. As a result, the ledger remains pending and the settlement has not been completed.The AI only explains the verified investigation result instead of independently guessing the transaction state.

AI-Powered Settled Agent

In our Solution, AI has the following responsibilities:

- Understand the merchant's settlement question
- Use structured backend investigation results
- Explain status and reason in simple language
- Highlight uncertainty
- Avoid inventing missing events

Exception Handling

If evidence is missing or contradictory:

Unknown + Exception : Instead of making a false claim, the agent tells the merchant that the available evidence is insufficient.

The evidence-first AI design reduces hallucination risk and makes the system easier to explain and audit.

Implementation and Technology Stack

Backend:

- Python
- Fast APIs
- Modular Service Architecture
- Transaction and Investigation APIs
- API: REST + CORS

AI

- OpenRouter LLM API
- Structured Prompting
- Exception-aware Responses

Engineering:

- Automated testing and Timestamp normalization
- Transaction Matching
- Docker-based Execution

Data:

- gateway.csv
- bank.csv
- ledger.csv
- PostgreSQL (Neon)

Frontend:

- Simple Merchant Chat
- Form Interface
- Transaction Status
- Investigation View
- React + Vite

The backend is organized into API, service, model, schema, database, utility and testing layers.

Reliabiliity of our Solution

- LLM is not the source of truth.
- Deterministic Transaction Matching may occur.
- Evidence-based reconciliation.
- Explicit Exception Handling is ensured.
- Timestamp Normalisation is applied.
- Audit-friendly investigation results is ensured.

Current Limitations

- It uses mock gateway, bank and ledger data.
- No live payment-provider integrations is done.
- Simplified settlement rules is applied.
- Our prototype is only for demonstration/workflow validation.

Future Scopes

Following features may be integrated in future in order to make it more practical and applicable:

- **Real APIs** → For Gateway and Banking integration.
- **Proactive Alerts** → To Notify merchants about delays.
- **Analytics** → **For** Historical settlement-failure trends.
- **Security** → For Authentication, RBAC and audit trails.
- **Escalation** → **For H**uman intervention for unresolved/high-risk cases.

What we Automate?

Merchant Question

↓

Transaction Tracing

↓

Reconciliation + Anomaly Detection

↓

AI Reply and Clear Merchant Answer

Expected Impact

For Merchants :

- Faster settlement answers
- No manual log investigation
- Simple explanations
- Greater transparency

For Operations :

- Reduced repetitive investigation
- Consistent process
- Faster anomaly identification
- Better scalability

Sample Demo (Processes) :

- Enter transaction ID
- Trace all three records
- Show reconciliation
- Show anomaly detection
- Display final status
- Demonstrate an exception case

Our Thinking

We do not ask AI to guess what happened.
Rather,
We give it the evidence and let it explain.

Regards,

Team Zero/Zero

THANK YOU!

05 September 2026